CAA 1

E-Commerce

Alejandro Pérez Bueno

Oct 14, 2026

Table of Contents

Section 1 2
Who needs a certified public key 2
1) Generation and sending of the PROPOSAL 2
2) Reading of the PROPOSAL by an owner 3
3) Generation and sending of a VOTE 3
4) Counting the VOTE documents 4
Section 2 5
(i) Vote linked to the proposal 5
(ii) Confidentiality until the deadline 5
(iii) Everyone can check the votes of the rest 5
References 7

Section 1

To solve this we need a way for each owner to commit to a vote before the deadline without anyone being able to read it, not even the administrator. Since all comunication between the administrator and the owners happens by email every message below travels over an insecure channel were anyone with access to the mail servers could read or modify it.

My solution is basicaly a commit and reveal scheme. Every owner encrypts their vote with an AES key that only they know, signs it and sends it before the deadline. Once the voting period is over they send the key so the vote can be opened and counted. Before the deadline the administrator holds a signed vote that nobody can read, and after it anyone can open the vote but nobody can change it. I chose this over sending the vote inside a digital envelope for the administrator becuase with an envelope he could open the vote the moment it arrives. The owner would have no real guarantee for requirement (ii) and would just have to trust him.

Let's go over who needs a certificate first and then each of the four steps.

Who needs a certified public key

Only the owners $A_1, \dots, A_n$. Every owner signs their vote and both the administrator and the rest of the owners must be able to verify those signatures, so each A_i needs a key pair $(Pub_i, Priv_i)$ and a digital certificate $Cert_i$ issued by a CA that everyone in the community trusts. In practice this could be the certificate in the DNIe or one issued by the FNMT wich most people in Spain already have, so there is no need to set up a PKI just for the building.

The administrator doesn't need one. He never signs anything and nobody encrypts anything for him with a public key so a certificate for him would not help with any of the three requirements.

> Design note
>
> IN a real system the administrator would probably sign the PROPOSAL so the owners know it really comes from him. However none of the three requirements ask for the authenticity of the proposal and the statement says that operations that do not serve any of them will be penalised. That is why I left it out. Section 2 explains why requirement (i) still holds without it.

1) Generation and sending of the PROPOSAL

The administrator writes the text of the proposal and adds three fields to it: a unique identifier ID_P (for instance a sequential number plus the year), the deadline T and a reveal window W that I explain in step 4. The identifier is needed because if the community votes the same text twice (for example "approve this year's budget" after a first rejection) both proposals would otherwise have the same hash and a vote from the first round could be replayed in the second one.

$$P = ID_P \| \text{text} \| T \| W$$

Where $\|$ is concatenation. P does not need to be secret or signed so the administrator just emails it in plaintext to every owner and keeps $H(P)$ to check the votes later.

2) Reading of the PROPOSAL by an owner

The owner reads P like any other email. The only cryptographic operation is computing its hash over the exact document received. They will need it later to tie the vote to this specific proposal:

$$h_P = H(P)$$

If the hash they compute is different from the one published at the end the owner will know they were sent somethign different. But they cannot prove who changed it since P is not signed and the administrator could always say that the owner edited their own copy.

3) Generation and sending of a VOTE

Owner A_i decides their vote v_i (either `for` or `against`) and does the following:

1. Generates a random AES key K_i. This key is used only for this vote and does not leave the owner's computer before the deadline.
2. Encrypts the vote encoded as a single fixed-length AES block so that `for` and `against` give ciphertexts of the same size: $C_i = AES_{K_i}(v_i)$
3. Signs the hash of the proposal together with the encrypted vote: $S_i = RSA_{Priv_i}(H(h_P \| C_i))$
4. Sends the VOTE document to the administrator before T:

$$VOTE_i = h_P \| C_i \| S_i \| Cert_i$$

Note that the signature is computed over the ciphertext C_i and not over v_i. Signing $H(h_P \| v_i)$ instead would leak the vote because there are only two possible values: anyone can apply RSA_{Pub_i} to the signature to recover the hash and compare it with $H(h_P \| \texttt{for})$ and $H(h_P \| \texttt{against})$. Signing the ciphertext avoids this as K_i is random and C_i says nothing about the vote.

When the VOTE arrives the administrator verifies $Cert_i$ following the steps from Module 2 (Herrera Joancomartí 2022): the signature of the CA, the validity period, that it does not appear in the CRL and that the identity in the certificate is one of the owners int he community's register. Then he extracts Pub_i and checks that $RSA_{Pub_i}(S_i) = H(h_P \| C_i)$ and that $h_P = H(P)$. He also stores the time the email arrived. At this point the administrator knows that owner A_i has voted but not what.If an owner sends more than one VOTE before T only the last valid one is kept.

As for which shared key to use there is no negociation here. K_i is not a key for a confidential channel between the owner and the administrator, it works more like a seal. The owner picks it alone and the administrator simply learns it when the owner reveals it in the next step. This is also why there is no digital envelope anywhere in the system since is never necessary to encrypt a message with a public key.

4) Counting the VOTE documents

Once T has passed the counting is done in three steps.

Key reveal

During the window W (which starts at T and lasts for instance 48 hours) each owner who voted sends their key to the administrator:

$$KEY_i = h_P \| K_i$$

It can go in plaintext because after the deadline the vote no longer has to be secret, in fact requirement (iii) asks for the opposite. It does not need a signature either: a key that is not the real one will not decrypt C_i into `for` or `against` so a forged one is detected straight away. The administrator does not publish anything untill W closes.

Verification and count

For every $VOTE_i$ that arrived before T and passed the checks from step 3 the administrator:

1. Decrypts the vote: $v_i = AES_{K_i}^{-1}(C_i)$
2. Checks that v_i is exactly `for` or `against`. If it is not, the key is wrong and he waits for the rigth one. If the owner never sends it within W, the vote counts as an abstention.
3. Adds v_i to the corresponding total.

Publication

The administrator emails every owner the result together with everything needed to check it:

$$RESULT = P \| \{h_P, C_i, S_i, Cert_i, K_i, v_i\}_{i=1\ldots n} \| \text{totals}$$

Each owner can then repeat the verification and the count for every other owner on their own computer and check that their own vote is in the list. Since RESULT is a normal email sent to each owner separately the owners also compare $H(RESULT)$ with each other (for example in the building's group chat) to make sure all of them received the same list.

Section 2

(i) Vote linked to the proposal

The link comes from the owner's signature. S_i is computed over $H(h_P \| C_i)$ so the hash of the proposal and the encrypted vote are signed as a single unit. If someone takes C_i and S_i and attaches them to a different proposal P' the verification would compute $H(H(P') \| C_i)$. This does not match the value inside the signature, so the check fails. Producing a new signature for P' is not posible without $Priv_i$ and only the owner has it.

Using the hash of the whole document instead of just a title or a number is also importamt. Since the hash function is collision free (Herrera Joancomartí 2022) it is unfeasible to write a different proposal P' with $H(P') = H(P)$. That way the administrator cannnot change the wording after the votes are cast and claim the owners voted on the new text. It is basically the same problem as the self-evaluation exercise in Module 2 where a hash with collisions lets someone swap a contract for 10 euros with one for 100. The unique identifier inside P covers the remaining case (the same text voted twice) as the two documents will never have the same hash.

As mentioned in Section 1 all of this holds even though the administrator does not sign P. If an owner received a fake proposal their vote would be linked to that fake one and rejected during the count, it could never end up counting for the real proposal. Furthermore the RESULT includes P and every h_P so a mismatch is visible to everyone.

(ii) Confidentiality until the deadline

Before T the only thing about the vote that leaves the owner's computer is C_i and it is encrypted with AES under a key that only the owner knows. Not even the administrator can read it, and this is the main difference with a digital envelope addressed to him: the owner does not need to trust anyone since they hold the guarantee themselves by deciding whem K_i is released. The signature doesn't leak anything either. Recovering the hash from S_i only gives $H(h_P \| C_i)$ which depends of the ciphertext and not of the vote, so the trick with the two possible values from step 3 does not work.

Once the owner sends K_i the vote stops being confidential but by then the deadline has passed and that is exactly the period the requirement covers. The scheme also stops the owner from cheating in the other direction. C_i is a single fixed-length block that is fixed by S_i before the deadline, and finding a different key that decrypts it into the opposite vote is unfeasible with AES. So an owner cannot wait to see how the others voted and then reveal a different value. The most they can do is not reveal K_i at all and that only turns their vote into an abstention as explained in step 4.

(iii) Everyone can check the votes of the rest

The RESULT messsage contains the signed encrypted vote, the certificate and the key of each owner. With that any owner can:

- Verify $Cert_i$ against the CA to be sure that Pub_i really belongs to A_i.

- Check S_i to confirm that A_i and nobody else produced C_i for this proposal and that it was not modified afterwards.
- Decrypt C_i with K_i and confirm that the published v_i is correct.

This means owners do not need to trust the administrator's count because they can recompute the totals themselves. Because the signatures also give non-repudiation an owner cannot claim later that they voted something different. If they signed with the DNIe this even holds legally, since a digital signature has the same legal value as a handwritten one (Soriano Ibáñez 2022). The administrator cannot add or change a vote in the name of an owner without their private key either.

The only thing he could still do is leave out a vote he received. The affected owner would notice it is missing from their copy of RESULT and could forward their signed VOTE to the rest of owners as proof. However RESULT is not signed and each owner gets their own email. The administrator could leave B's vote out only from A's copy and B would never find out. That is why the owners compare $H(RESULT)$ with each other as mentioned in step 4: if all the hashes match everyone is checking the same list.

Without certificates from a CA that all owners trust none of this would work. Nobody could be sure that Pub_i belongs to A_i and the administrator could slip in a fake vote signed with a key pair of his own.

All in all, the protocol covers the three requirements with one signature and one AES encryption per owner. The one thing left out on purpose is the arrival time of a VOTE. It comes from the email server so a dishonest administrator could claim a vote arrived late. A timestamp from a trusted authority would fix this, but none of the three requirments asks for it.

References

Herrera Joancomartí, Jordi. 2022. *Security in Electronic Commerce*. Fundació Universitat Oberta de Catalunya.

Soriano Ibáñez, Miquel. 2022. *Introduction to Electronic Commerce*. Fundació Universitat Oberta de Catalunya.